

GitHub Actions Build Guide for ARM Mac

Version: 0.6.0

Date: May 17, 2026

Purpose: Build ARM Mac binaries using GitHub's cloud runners

What is GitHub Actions?

GitHub Actions is a CI/CD service that runs automated workflows on GitHub's servers. For our purposes, it allows you to:

- Build ARM Mac binaries without owning an ARM Mac
- Build both Intel and ARM versions simultaneously
- Automate builds on every commit or manually trigger them
- Download the built executables as artifacts

Cost: Free for public repositories (2,000 minutes/month for private repos)

Prerequisites

1. **GitHub Account** - Free account at <https://github.com>
 2. **Git Installed** - Check with `git --version`
 3. **Your Code Uploaded** - Repository must be on GitHub
-

Step 1: Create a GitHub Repository

Option A: New Repository (Recommended)

1. Go to <https://github.com/new>
2. Repository name: `claude-terminal` (or any name you prefer)
3. Description: "Desktop GUI wrapper for Claude CLI"
4. Choose **Public** (free unlimited Actions minutes) or **Private**
5. **DO NOT** initialize with README, .gitignore, or license (we have those)
6. Click "Create repository"

Option B: Use Existing Repository

If you already have a repo, skip to Step 2.

Step 2: Upload Your Code to GitHub

First-Time Setup

```
# Navigate to your Tauri Builds directory
cd "C:\Users\joshua.gates\Dev Projects\Claude Terminal Project\claude-desktop-app\Tauri Builds"

# Initialize git (if not already done)
git init

# Add GitHub repository as remote (replace YOUR_USERNAME)
git remote add origin https://github.com/YOUR_USERNAME/claude-terminal.git

# Create .gitignore to exclude build artifacts
cat > .gitignore << 'EOF'
# Dependencies
node_modules/
src-tauri/target/

# Build outputs
dist/
dist-electron/
src-tauri/target/release/

# OS files
.DS_Store
Thumbs.db

# IDE
.vscode/
.idea/

# Logs
*.log
npm-debug.log*

# Temporary files
*.tmp
*.temp
.claude/
.claude-desktop/

# Archives
*.zip
*.tar.gz
*.dmg
EOF

# Stage all files
git add .

# Create first commit
git commit -m "Initial commit - Claude Terminal v0.6.0"

# Push to GitHub
git push -u origin master
```

Note: If you get an authentication error, you may need to:

- Use a Personal Access Token instead of password
- Go to GitHub Settings → Developer settings → Personal access tokens → Tokens (classic)
- Generate new token with `repo` scope
- Use token as password when pushing

Step 3: Create GitHub Actions Workflow

GitHub Actions workflows are defined in YAML files in `.github/workflows/` directory.

Create the Workflow File

```
# Still in Tauri Builds directory
mkdir -p .github/workflows

# Create the workflow file (content below)
```

File: `.github/workflows/build.yml`

```
name: Build Claude Terminal

on:
  # Trigger on push to master branch
  push:
    branches: [ master, main ]

  # Trigger on pull requests
  pull_request:
    branches: [ master, main ]

  # Allow manual trigger from Actions tab
  workflow_dispatch:

jobs:
  # Windows Build
  build-windows:
    runs-on: windows-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'
```

- name: Setup Rust
uses: dtolnay/rust-toolchain@stable
- name: Install dependencies
run: npm install
- name: Build Windows app
run: npm run tauri:build:win
- name: Upload Windows artifact
uses: actions/upload-artifact@v4
with:
 - name: claude-terminal-windows
 - path: |
 - src-tauri/target/release/bundle/nsis/*.exe
 - src-tauri/target/release/bundle/msi/*.msi

Mac Intel Build

build-mac-intel:

runs-on: macos-13 # Intel Mac

steps:

- name: Checkout code
uses: actions/checkout@v4
- name: Setup Node.js
uses: actions/setup-node@v4
with:
 - node-version: '18'
- name: Setup Rust
uses: dtolnay/rust-toolchain@stable
with:
 - targets: x86_64-apple-darwin
- name: Install dependencies
run: npm install
- name: Build Mac Intel app
run: npm run tauri:build:mac:x64
- name: Upload Mac Intel artifact
uses: actions/upload-artifact@v4
with:
 - name: claude-terminal-mac-intel
 - path: |
 - src-tauri/target/release/bundle/dmg/*.dmg
 - src-tauri/target/release/bundle/macos/*.app.tar.gz

Mac ARM Build (M1/M2/M3)

build-mac-arm:

runs-on: macos-14 # ARM Mac (M1)

```

steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: '18'

  - name: Setup Rust
    uses: dtolnay/rust-toolchain@stable
    with:
      targets: aarch64-apple-darwin

  - name: Install dependencies
    run: npm install

  - name: Build Mac ARM app
    run: npm run tauri:build:mac:arm64

  - name: Upload Mac ARM artifact
    uses: actions/upload-artifact@v4
    with:
      name: claude-terminal-mac-arm
      path: |
        src-tauri/target/release/bundle/dmg/*.dmg
        src-tauri/target/release/bundle/macos/*.app.tar.gz

```

Add Scripts to package.json

Make sure these build scripts exist in your `package.json`:

```

{
  "scripts": {
    "tauri:build:win": "tauri build --target x86_64-pc-windows-msvc",
    "tauri:build:mac:x64": "tauri build --target x86_64-apple-darwin",
    "tauri:build:mac:arm64": "tauri build --target aarch64-apple-darwin"
  }
}

```

Step 4: Push Workflow to GitHub

```

# Add workflow file
git add .github/workflows/build.yml

# Commit
git commit -m "Add GitHub Actions build workflow"

```

```
# Push to GitHub
git push
```

Step 5: Trigger a Build

Automatic Trigger (Recommended)

The workflow automatically runs when you push commits to master/main branch:

```
# Make any change (or use --allow-empty)
git commit --allow-empty -m "Trigger build"
git push
```

Manual Trigger

1. Go to your repository on GitHub
 2. Click the **Actions** tab
 3. Click **Build Claude Terminal** workflow (left sidebar)
 4. Click **Run workflow** button (right side)
 5. Select branch (usually **master**)
 6. Click **Run workflow**
-

Step 6: Monitor the Build

1. Go to **Actions** tab on GitHub
2. Click on the running workflow
3. You'll see three jobs running in parallel:
 - **build-windows**
 - **build-mac-intel**
 - **build-mac-arm**
4. Click on any job to see live logs
5. Wait for all jobs to complete (usually 10-20 minutes)

Expected Times:

- Windows: ~8-12 minutes
 - Mac Intel: ~10-15 minutes
 - Mac ARM: ~10-15 minutes
-

Step 7: Download Built Artifacts

Once builds complete successfully:

1. On the workflow run page, scroll down to **Artifacts** section
2. You'll see three downloadable artifacts:

- `claude-terminal-windows` (Windows .exe installer)
 - `claude-terminal-mac-intel` (Intel Mac .dmg)
 - `claude-terminal-mac-arm` (ARM Mac .dmg)
3. Click each to download as .zip file
 4. Extract the .zip to get your executables

Artifact Retention: GitHub keeps artifacts for 90 days by default.

Step 8: Distribute the Builds

After extracting artifacts, you'll have:

```
claude-terminal-windows.zip
└─ Claude_Terminal_0.6.0_x64-setup.exe  (~12MB)

claude-terminal-mac-intel.zip
└─ Claude_Terminal_0.6.0_x64.dmg        (~6MB)

claude-terminal-mac-arm.zip
└─ Claude_Terminal_0.6.0_aarch64.dmg   (~6MB)
```

For Users:

- **Windows:** Run the .exe installer
 - **Mac Intel:** Open the x64.dmg, drag to Applications
 - **Mac ARM (M1/M2/M3):** Open the aarch64.dmg, drag to Applications
-

Troubleshooting

Build Fails: "npm install" errors

Problem: Missing dependencies or version conflicts

Solution: Check the build logs and ensure package.json has correct versions. You can also add cache steps:

```
- name: Cache node modules
  uses: actions/cache@v3
  with:
    path: node_modules
    key: ${{ runner.os }}-node-${{ hashFiles('package-lock.json') }}
```

Build Fails: "Rust compilation error"

Problem: Rust toolchain or dependency issues

Solution: Check Cargo.toml dependencies and ensure all are compatible with the target platform.

Build Fails: "Permission denied"

Problem: Scripts not executable

Solution: Add permission step before running scripts:

```
- name: Make scripts executable
  run: chmod +x scripts/*.sh
```

Artifacts Not Uploaded

Problem: Build succeeded but no artifacts appear

Solution: Check the artifact path matches your actual build output:

```
# Locally verify the build output location
npm run tauri:build:win
ls -R src-tauri/target/release/bundle/
```

Update the **path:** in workflow to match actual output location.

"Resource not accessible by integration" Error

Problem: GitHub Actions permissions issue

Solution:

1. Go to repository Settings
2. Click **Actions** → **General**
3. Under "Workflow permissions", select **Read and write permissions**
4. Click **Save**

Slow Builds

Problem: Each build takes 15-20 minutes

Solutions:

- Add dependency caching (see above)
- Only build on tagged releases instead of every commit
- Use **workflow_dispatch** for manual-only builds

```
on:
  workflow_dispatch: # Manual only
  push:
    tags:
      - 'v*' # Only on version tags like v0.6.0
```

Advanced: Build Only on Release

If you don't want to build on every commit, use this trigger:

```
on:
  # Only build when you create a release
  release:
    types: [created]

# Keep manual trigger available
workflow_dispatch:
```

Then create releases via GitHub UI:

1. Go to repository
2. Click **Releases** → **Draft a new release**
3. Tag: `v0.6.0`, Title: `Version 0.6.0`
4. Click **Publish release**
5. Build automatically starts

Cost Estimate

Public Repository: Free unlimited minutes

Private Repository:

- 2,000 free minutes/month
- Each build uses ~30-40 minutes total (all 3 platforms)
- ~50 builds/month within free tier
- After free tier: \$0.008/minute (Windows), \$0.08/minute (Mac)

Recommendation: Use public repository or build manually when needed.

Alternative: Build Locally, Push Artifacts

If you prefer not to use GitHub Actions:

1. Build Windows on your PC: `npm run tauri:build:win`
 2. Ask a friend with ARM Mac to build: `npm run tauri:build:mac:arm64`
 3. Manually upload .exe and .dmg files to GitHub Releases
-

Summary Checklist

- ☐ Create GitHub repository (public recommended)
- ☐ Push code to GitHub
- ☐ Create `.github/workflows/build.yml`

- ☐ Ensure build scripts exist in package.json
 - ☐ Push workflow to GitHub
 - ☐ Trigger build (manual or automatic)
 - ☐ Monitor build progress in Actions tab
 - ☐ Download artifacts when complete
 - ☐ Distribute executables to users
-

Next Steps

Once builds are working:

1. **Automate Releases** - Automatically create GitHub releases with attached binaries
2. **Code Signing** - Sign Windows .exe and Mac .dmg for better user experience
3. **Auto-Updates** - Implement Tauri's updater for in-app updates

See [BUILD_INSTRUCTIONS_v0.6.0.md](#) for more details on distribution.

Questions? Check the workflow logs first - they're very detailed and usually point to the exact issue.

Need Help? GitHub Actions documentation: <https://docs.github.com/en/actions>